

+Café

Encontro 2

Back-end Supabase, modelagem e segurança

Estudo dirigido independente - carga horária de 4 horas

Público-alvo	Alunos iniciantes em programação e desenvolvimento mobile.
Tecnologias	Linux, VS Code, React Native, Expo SDK 54, TypeScript e Supabase.
Resultado	Entrega prática do Encontro 2 do aplicativo +Café.

Material para aula prática com exercícios e gabarito orientativo

Como utilizar este estudo dirigido

Este PDF foi preparado para funcionar de forma independente. O monitor apresenta o conceito, demonstra a implementação, acompanha a prática dos alunos e encerra cada bloco com uma validação. Os exercícios devem ser realizados antes da consulta ao gabarito.

Objetivos de aprendizagem

- Compreender tabelas, linhas, colunas e relacionamentos.
- Ler e discutir o Diagrama Entidade-Relacionamento do +Café.
- Criar tabelas, chaves primárias, chaves estrangeiras e restrições.
- Executar scripts SQL, triggers, views e dados iniciais.
- Aplicar Row Level Security para separar permissões de Gerente, Garçom e Cliente.

Organização das 4 horas

Bloco	Duração sugerida	Dinâmica
1	60 min	Conceito, preparação e primeira entrega.
2	60 min	Implementação guiada e explicação do código.
3	60 min	Integração, testes e correções.
4	60 min	Desafio, exercícios, revisão e checkpoint.

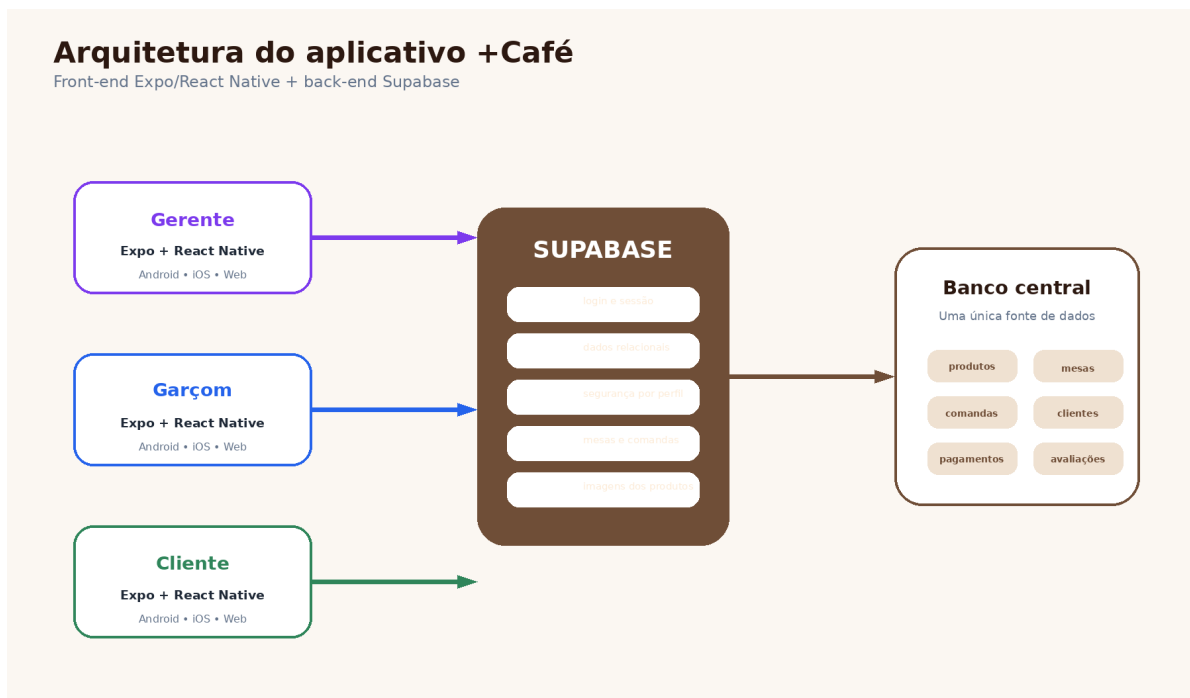
Orientação: o monitor deve executar previamente os comandos e manter um projeto de referência aberto para comparação.

Encontro 2 — Back-end Supabase, modelagem e segurança

Tempo	Atividade	Entrega esperada
0:00–0:45	Conceitos de banco relacional e DER	Modelo de dados compreendido
0:45–1:45	Criar tabelas, restrições e índices	Esquema executado no Supabase
1:45–2:30	Popular categorias, produtos e mesas	Dados de teste visíveis
2:30–3:20	Auth, perfis e papéis	Usuários gerente, garçom e cliente
3:20–4:00	RLS e testes de permissão	Acesso protegido por perfil

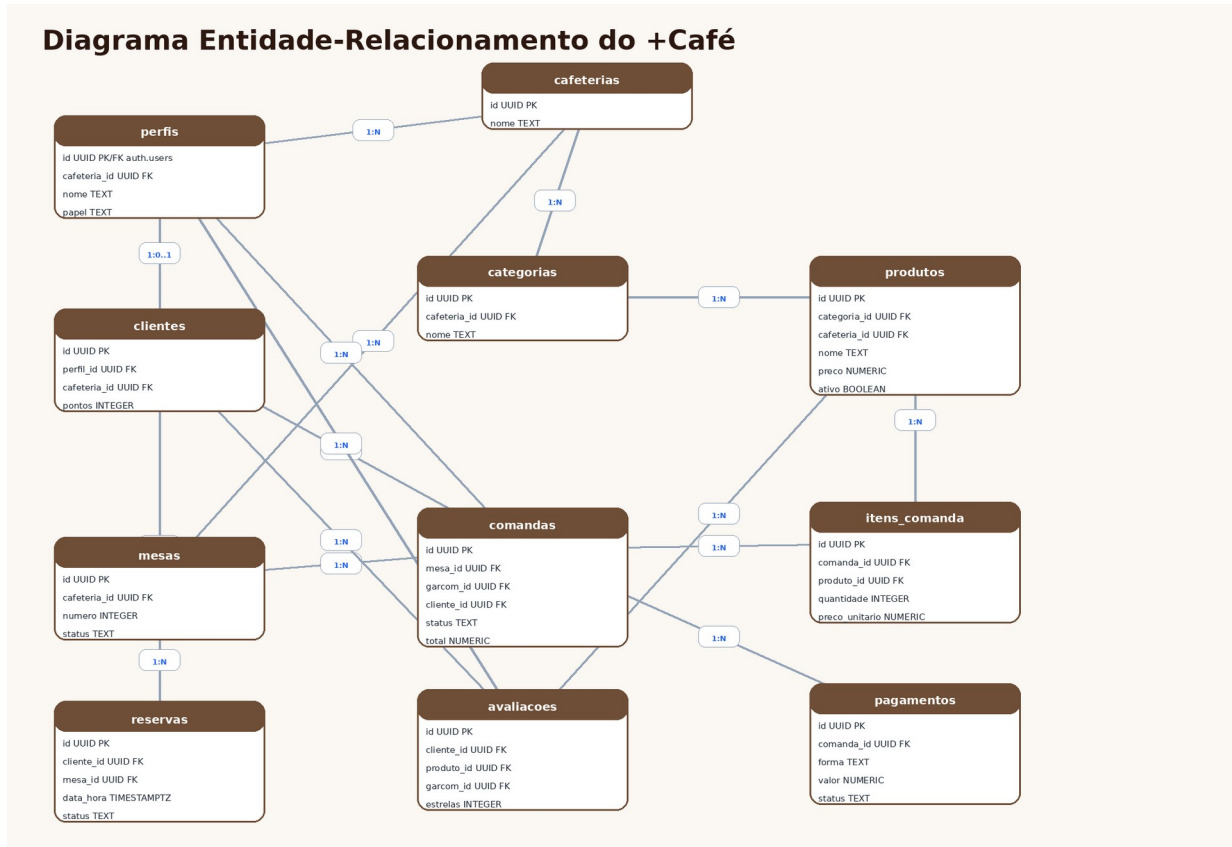
1. O Supabase como back-end

Neste projeto, back-end não significa obrigatoriamente criar um servidor Node.js. O Supabase fornece PostgreSQL, Auth, API, RLS, Realtime e Storage. A lógica sensível permanece no banco por meio de restrições, políticas, triggers e funções.



Arquitetura centralizada: todos os aparelhos consultam o mesmo back-end.

2. Modelagem relacional



DER do +Café: entidades, chaves primárias e estrangeiras.

- PK identifica uma linha de forma única.
- FK conecta tabelas e impede referências inexistentes.
- 1:N significa que uma linha do lado 1 pode ter várias no lado N.
- CHECK limita estados e valores válidos.
- Índices aceleram filtros usados com frequência.

3. Executando o esquema

1. Abra o SQL Editor do Supabase.
2. Execute 001_schema.sql.
3. Execute 002_functions_views.sql.
4. Execute 003_rls.sql.
5. Execute 004_seed.sql.

Exemplo de tabela com regras:

```
create table public.mesas (
  id uuid primary key default gen_random_uuid(),
  cafeteria_id uuid not null references public.cafeterias(id),
  numero integer not null check (numero > 0),
  status text not null default 'livre'
  check (status in ('livre', 'ocupada', 'reservada', 'limpando')),
  unique (cafeteria_id, numero)
);
```

4. Trigger e view

A trigger recalcula o total quando um item é incluído, alterado ou removido. A view resume indicadores para o gerente. Assim, regras importantes não dependem apenas do celular.

```
update public.comandas
set total = coalesce((
  select sum(quantidade * preco_unitario)
  from public.itens_comanda
  where comanda_id = v_id
), 0)
where id = v_id;
```

5. Papéis e criação de usuários

- Cliente: cadastro público pelo app.
- Gerente e garçom: crie usuários no painel Auth e promova com 005_promover_usuarios.sql.
- O papel fica em public.perfis, não em um campo livre da interface.

6. RLS — segurança por linha

RLS responde: “este usuário pode ler ou alterar esta linha?”. A publishable key pode ficar no aplicativo somente porque as políticas limitam o acesso.

```
create policy produtos_gerente
on public.produtos for all to authenticated
using (
  cafeteria_id = public.minha_cafeteria_id()
  and public.meu_papel() = 'gerente'
)
with check (
  cafeteria_id = public.minha_cafeteria_id()
  and public.meu_papel() = 'gerente'
);
```

Teste obrigatório

Entre com cliente e tente inserir produto: deve falhar. Entre com gerente e repita: deve funcionar. Segurança só está comprovada quando a tentativa indevida é bloqueada.

Exercícios e atividades práticas

6. Identifique três relações 1:N no DER.
7. Crie a categoria “Bebidas geladas”.
8. Explique a diferença entre restrição CHECK e política RLS.
9. Teste se um cliente consegue alterar o papel do próprio perfil.

Desafios adicionais

10. Crie uma restrição para impedir preços negativos na tabela de produtos.
11. Explique por que o campo cafeteria_id deve aparecer nas tabelas operacionais.

Apêndice A - Gabarito orientativo

Respostas das atividades originais: Exemplos 1:N: categoria-produtos, mesa-comandas e comanda-itens. CHECK valida o conteúdo da linha; RLS decide quem pode acessar a linha.

Respostas dos desafios adicionais

12. Use CHECK (preço >= 0) na definição da coluna ou em uma alteração de tabela.
13. cafeteria_id permite isolar os dados de cada estabelecimento e aplicar políticas RLS por cafeteria.

Apêndice B - Checklist de conclusão

- ☐ DER foi compreendido e discutido pela turma.
- ☐ Tabelas foram criadas sem erros no SQL Editor.
- ☐ Chaves estrangeiras preservam a integridade dos relacionamentos.
- ☐ Dados iniciais aparecem no Table Editor.
- ☐ Usuário Cliente não consegue administrar produtos.
- ☐ Usuário Gerente consegue executar operações autorizadas.

Apêndice C - Erros comuns

Problema	Correção provável
Could not parse JSON returned from npm pack	Verifique npm --version. Se estiver no npm 12, instale npm@11 e repita com SDK 54.
Invalid API key / Failed to fetch	Confira URL e publishable key no .env e reinicie npx expo start --clear.
new row violates row-level security policy	A política não permite a ação ou cafeteria_id não corresponde ao perfil.
Perfil não encontrado após cadastro	Confirme o trigger handle_new_user e o cafeteria_id enviado no metadata.
Realtime não atualiza	Adicione a tabela à publicação supabase_realtime e confirme a política SELECT.
QR não abre câmera	Conceda permissão; teste em dispositivo físico; a câmera pode ter limitações em iframe na web.
Build gera AAB, não APK	Use o perfil preview com android.buildType = apk.

Referências oficiais

- Expo. Create Expo App e compatibilidade do Expo Go durante a transição do SDK 57.
<https://docs.expo.dev/more/create-expo/>
- Expo. Referência do SDK 54 e versões do React Native, React e Node.js.
<https://docs.expo.dev/versions/v54.0.0/>

- Expo. Expo Router. <https://docs.expo.dev/versions/v54.0.0/sdk/router/>
- Expo. Camera e leitura de códigos. <https://docs.expo.dev/versions/v54.0.0/sdk/camera/>
- Expo. EAS Build e geração de APK. <https://docs.expo.dev/build-reference/apk/>
- Supabase. Expo React Native Quickstart.
<https://supabase.com/docs/guides/getting-started/quickstarts/expo-react-native>
- Supabase. Password-based Auth. <https://supabase.com/docs/guides/auth/passwords>
- Supabase. Row Level Security. <https://supabase.com/docs/guides/database/postgres/row-level-security>
- Supabase. Securing the Data API. <https://supabase.com/docs/guides/api/securing-your-api>
- Supabase. Realtime database changes. <https://supabase.com/docs/guides/realtime/subscribing-to-database-changes>

Nota de atualização

As ferramentas evoluem. Antes da aula, o monitor deve executar expo-doctor, revisar os avisos do Expo SDK e conferir as páginas oficiais de Auth, RLS, Realtime e EAS Build.